XChain Provider 인터페이스 및 구현 분석

개요

`lib/xchain/provider` 패키지는 Omni 네트워크의 크로스체인 데이터 제공자 인터페이스와 구현을 담당합니다. 이 패키지는 다양한 체인으로부터 크로스체인 블록과 이벤트 로그를 수집하고, 비동기 스트리밍을 통해 실시간으로 데이터를 제공하는 핵심 컴포넌트입니다.

Provider 인터페이스

핵심 인터페이스 정의

```
type Provider interface {

// 비동기 스트리밍 - 고루틴에서 영원히 실행

StreamAsync(ctx context.Context, req ProviderRequest, callback ProviderCallback)

// 동기 스트리밍 - 첫 번째 콜백 오류 시 반환

StreamBlocks(ctx context.Context, req ProviderRequest, callback ProviderCallback

// EVM 이벤트 로그 스트리밍

StreamEventLogs(ctx context.Context, req EventLogsReq, callback EventLogsCallbac

// 단일 블록 조회

GetBlock(ctx context.Context, req ProviderRequest) (Block, bool, error)

// 커서 조회 메서드들

GetSubmittedCursor(ctx context.Context, ref Ref, stream StreamID) (SubmitCursor,

GetEmittedCursor(ctx context.Context, ref Ref, stream StreamID) (EmitCursor, boo
```

```
// 유틸리티 메서드들
ChainVersionHeight(ctx context.Context, chainVer ChainVersion) (uint64, error)
GetSubmission(ctx context.Context, chainID  uint64, txHash common.Hash) (Submiss
}
```

요청 구조체

ProviderRequest

```
type ProviderRequest struct {
ChainID  uint64  // 조회할 소스 체인 ID
Height  uint64  // 조회 시작 높이 (포함)
ConfLevel ConfLevel // 확인 레벨 보장
}
```

EventLogsReq

```
type EventLogsReq struct {
ChainID  uint64  // 소스 체인 ID
Height  uint64  // 조회 높이
ConfLevel ConfLevel // 확인 레벨
FilterAddresses []common.Address // 주소 필터 (0개 이상)
FilterTopics []common.Hash // 토픽 필터 (첫 번째 위치만)
```

```
}
```

콜백 함수 타입

```
type ProviderCallback func(context.Context, Block) error

type EventLogsCallback func(ctx context.Context, header *types.Header, events []
```

Provider 구현체

구조체 정의

```
type Provider struct {

network netconf.Network // 네트워크 설정

ethClients  map[uint64]ethclient.Client // 체인별 RPC 클라이언트

cChainID  uint64  // 합의 체인 ID

cProvider cchain.Provider // 합의 체인 제공자

backoffFunc  func(context.Context) func() // 백오프 함수

mu sync.Mutex

confHeads  map[xchain.ChainVersion]uint64  // 체인 버전별 헤드 캐시

}
```

초기화 및 설정

New 함수

```
func New(network netconf.Network, rpcClients map[uint64]ethclient.Client, cPro
```

백오프 설정:

- 최대 지연: 10초
- 지수 백오프 알고리즘 사용
- 모든 EVM 체인에 동일 적용

워커 수 결정:

```
var fetchWorkerThresholds = []struct {
Workers uint64 // 워커 수
MinPeriod time.Duration // 최소 임계값
}{
{Workers: 1, MinPeriod: time.Second}, // 일반 체인용 1개 워커
{Workers: 2, MinPeriod: time.Second / 2}, // 빠른 체인용 2개 워커 (op_sepolia)
{Workers: 4, MinPeriod: 0}, // 가장 빠른 체인용 4개 워커 (arb_sepolia)
}
```

스트리밍 메커니즘

1. 비동기 스트리밍 (StreamAsync)

특징:

- 고루틴에서 영원히 실행
- 즉시 반환 (체인 ID 유효성 검사만 수행)

- 모든 fetch 및 콜백 오류에 대해 백오프와 함께 무한 재시도
- 컨텍스트 취소 시에만 종료

구현 플로우:

```
func (p *Provider) StreamAsync(ctx context.Context, req ProviderRequest, callbac
if _, ok := p.network.Chain(req.ChainID); !ok {
return errors.New("unknown chain ID")
}

go func() {
err := p.stream(ctx, req, callback, true) // retryCallback=true
if err != nil {
log.Error(ctx, "Streaming xprovider blocks failed unexpectedly [BUG]", err)
}
}()

return nil
}
```

2. 동기 스트리밍 (StreamBlocks)

특징:

- 블로킹 방식으로 실행
- fetch 오류는 백오프와 함께 무한 재시도

- 첫 번째 콜백 오류 시 즉시 반환
- 워커가 애플리케이션 오류 시 리셋해야 할 때 유용

구현:

```
func (p *Provider) StreamBlocks(ctx context.Context, req ProviderRequest, callba
return  p.stream(ctx, req, callback, false) // retryCallback=false
}
```

3. 공통 스트림 로직

stream 함수의 핵심 구성요소:

워커 설정

```
workers, err := getWorkers(chain)
if  err != nil {
return  err
}
```

시작 높이 결정

```
fromHeight := req.Height
if  fromHeight < chain.DeployHeight {
fromHeight = chain.DeployHeight
}
```

스트림 의존성 구성

```
deps := stream.Deps[xchain.Block]{
FetchWorkers: workers,
FetchBatch: func(ctx context.Context, height  uint64) ([]xchain.Block, error) {
// 블록 fetch 로직
},
Backoff: p.backoffFunc,
ElemLabel: "block",
HeightLabel: "height",
RetryCallback: retryCallback,
// ... 기타 설정
}
```

블록 조회 메커니즘

GetBlock 함수

합의 체인 처리:

```
if  req.ChainID == p.cChainID {
if  p.cProvider == nil {
return xchain.Block{}, false, errors.New("consensus provider not set")
}
b, ok, err := p.cProvider.XBlock(ctx, req.Height, false)
```

```
// ... 검증 로직
return b, true, nil
}
```

EVM 체인 처리:

1. **확인 캐시 검사**: `confirmedCache(req.ChainVersion(), req.Height)`
2. **헤더 조회**: 캐시 미스 시 최신 헤드 조회
3. **높이 검증**: 요청 높이가 체인 헤드보다 높으면 false 반환
4. **메시지 및 영수증 수집**: `getMsgsAndReceipts()`
5. **XBlock 구성**: 모든 데이터를 조합하여 반환

확인 레벨별 높이 조회

```
func heightByConfLevel(ctx context.Context, ethCl ethclient.Client, confLevel x
// 직접 조회 가능한 확인 레벨들
headers := map[xchain.ConfLevel]ethclient.HeadType{
xchain.ConfFinalized: ethclient.HeadFinalized,
}
if headType, ok := headers[confLevel]; ok {
header, err := ethCl.HeaderByType(ctx, headType)
return header.Number.Uint64(), nil
}
// MinX 확인 레벨들은 latest에서 지연값만큼 빼기
delay := confLevel.MinX()
```

```
latest, err := ethCl.BlockNumber(ctx)
return umath.SubtractOrZero(latest, delay), nil
}
```

메시지 및 영수증 수집

getMsgsAndReceipts 함수

이벤트 로그 필터링:

```
portalAbi, err := bindings.OmniPortalMetaData.GetAbi()
msgEvent := portalAbi.Events["XMsg"]
receiptEvent := portalAbi.Events["XReceipt"]

topics := []common.Hash{msgEvent.ID, receiptEvent.ID}
addrs := []common.Address{chain.PortalAddress}
events, err := getEventLogs(ctx, rpcClient, addrs, blockHash, topics)
```

이벤트 파싱:

```
for _, event := range events {
if event.Topics[0] == msgEvent.ID {
msg, err := parseXMsg(filterer, event, chainID)
msgs = append(msgs, msg)
```

```
} else if event.Topics[0] == receiptEvent.ID {
receipt, err := parseXReceipt(filterer, event, chainID)
receipts = append(receipts, receipt)
}
}
```

샤드 검증:

```
expectedShards := make(map[xchain.ShardID]bool)
for _, stream := range p.network.StreamsTo(chainID) {
expectedShards[stream.ShardID] = true
}

verifyShard := func(shardID xchain.ShardID) error {
if !expectedShards[shardID] {
return errors.New("unexpected shard")
}
return nil
}
```

이벤트 로그 스트리밍

StreamEventLogs 함수

특징:

- EVM 이벤트 로그를 실시간으로 스트리밍
- 주소 및 토픽 필터링 지원
- 순차적 높이로 콜백 호출 보장

구현 구조:

```
type events struct {
Header *types.Header
Events []types.Log
}

deps := stream.Deps[events]{
FetchBatch: func(ctx context.Context, height  uint64) ([]events, error) {
logs, header, exists, err := p.GetEventLogs(ctx, fetchReq)
if !exists {
return  nil, nil
}
return []events{{Header: header, Events: logs}}, nil
},
// ... 기타 설정
}
```

커서 관리

발행 커서 (EmitCursor) 조회

합의 체인:

```
if  stream.SourceChainID == p.cChainID {
block, err := getConsXBlock(ctx, ref, p.cProvider)
return xchain.EmitCursor{
StreamID: stream,
MsgOffset: block.Msgs[len(block.Msgs)-1].StreamOffset,
}, true, nil
}
```

EVM 체인:

```
caller, err := bindings.NewOmniPortalCaller(chain.PortalAddress, rpcClient)
offset, err := caller.OutXMsgOffset(opts, stream.DestChainID, uint64(stream.Shar
```

제출 커서 (SubmitCursor) 조회

```
msgOffset, err := caller.InXMsgOffset(callOpts, stream.SourceChainID, uint64(str
attestOffset, err := caller.InXBlockOffset(callOpts, stream.SourceChainID, uint6

return xchain.SubmitCursor{
StreamID: stream,
MsgOffset: msgOffset,
AttestOffset: attestOffset,
```

```
}, true, nil
```

참조 시스템 (Ref)

Ref 구조체

```
type Ref struct {
Height *uint64  // 절대 높이 지정 (nil이 아닌 경우)
ConfLevel *ConfLevel // 헤드 기준 상대 블록 지정 (nil이 아닌 경우)
}
```

미리 정의된 참조

```
var (
LatestRef = ConfRef(ConfLatest) // 최신 확인 레벨 참조
FinalizedRef = ConfRef(ConfFinalized) // 최종화 확인 레벨 참조
)
```

참조 생성 함수

```
func  ConfRef(level ConfLevel) Ref // 확인 레벨로 참조 생성
func  HeightRef(height  uint64) Ref // 높이로 참조 생성
```

성능 최적화

1. 헤드 캐싱

```
type Provider struct {
mu sync.Mutex
confHeads  map[xchain.ChainVersion]uint64  // 체인 버전별 헤드 캐시
}

func (p *Provider) confirmedCache(chainVer xchain.ChainVersion, height  uint64)
p.mu.Lock()
defer  p.mu.Unlock()
return  p.confHeads[chainVer] >= height
}
```

캐싱 효과:

- HeaderByType 쿼리 감소
- L2 배치 최종화 시 스트림 지연 최적화
- 네트워크 요청 최소화

2. 동적 워커 수 조정

체인 블록 주기에 따른 워커 수 결정:

- 느린 체인 (≥1초): 1개 워커
- 빠른 체인 (≥0.5초): 2개 워커

- 가장 빠른 체인 (<0.5초): 4개 워커

3. 백오프 전략

```
backoffFunc := func(ctx context.Context) func() {
const maxDelay = time.Second * 10
cfg := expbackoff.DefaultConfig
cfg.MaxDelay = maxDelay
return expbackoff.New(ctx, expbackoff.With(cfg))
}
```

오류 처리 및 복구

1. 재시도 메커니즘

fetch 오류:

- 지수 백오프와 함께 무한 재시도
- 최대 지연 시간: 10초
- RPC 제공자 로드 밸런싱 고려

콜백 오류:

- StreamAsync: 무한 재시도
- StreamBlocks: 즉시 반환

2. 검증 및 오류 감지

블록 검증:

```go
Verify: func(_ context.Context, block xchain.Block, h uint64) error {
if block.ChainID != req.ChainID {
return errors.New("invalid block source chain id")
}
if block.BlockHeight != h {
return errors.New("invalid block height")
}
return nil
}
```

이벤트 로그 검증:

```go
for i, log := range logs {
if i > 0 && log.Index <= logs[i-1].Index {
return nil, errors.New("unordered log index")
}
if log.BlockHash != blockHash {
return nil, errors.New("unexpected log block hash")
}
}
```

메트릭 및 모니터링

수집되는 메트릭

1. **스트림 지연 (streamLag)**: 블록 타임스탬프와 현재 시간의 차이
2. **fetch 오류 (fetchErrTotal)**: fetch 실패 횟수
3. **콜백 오류 (callbackErrTotal)**: 콜백 실패 횟수
4. **스트림 높이 (streamHeight)**: 현재 처리 중인 블록 높이
5. **콜백 지연 (callbackLatency)**: 콜백 실행 시간

메트릭 라벨

- **chainVersionName**: 체인 버전 이름
- **streamType**: “xblock” 또는 “event”

이러한 Provider 구현은 Omni 네트워크의 크로스체인 데이터 수집과 실시간 스트리밍을 안정적이고 효율적으로 처리하는 핵심 인프라를 제공합니다.